

# Phase 2



DLBDSPBDM01

Project: Build a Data Mart in SQL

Piotr Trybuś

92132989

# Phase 2 Objectives

- Implement the ER model from Phase 1 in a relational DBMS
- Create all entities and relationships using SQL
- Enforce primary and foreign key constraints
- Populate each table with at least 20 records
- Validate the model using SQL test cases

# Technical Environment

- Database Management System: DuckDB
- SQL Client: DBeaver
- Database Schema: booking

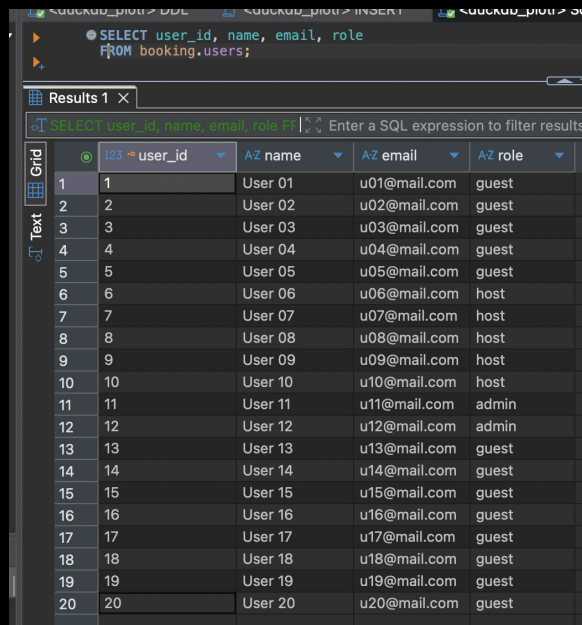
# Implemented ER Diagram

# Entity: users

- **Purpose:**  
Stores all system users (guests, hosts, admins)

```
● CREATE TABLE booking.users (  
  user_id INT NOT NULL,  
  name VARCHAR,  
  email VARCHAR,  
  role VARCHAR,  
  profile_picture VARCHAR,  
  created_at TIMESTAMP,  
  PRIMARY KEY (user_id)  
);
```

- **Test Case**



The screenshot shows a database management tool interface. At the top, there's a SQL query editor with the following query: `SELECT user_id, name, email, role FROM booking.users;`. Below the query, the results are displayed in a table grid. The table has four columns: `user_id`, `name`, `email`, and `role`. The results show 20 rows of data, with `user_id` ranging from 1 to 20. The `name` column contains 'User 01' through 'User 20'. The `email` column contains email addresses like 'u01@mail.com' through 'u20@mail.com'. The `role` column contains roles like 'guest', 'host', and 'admin'.

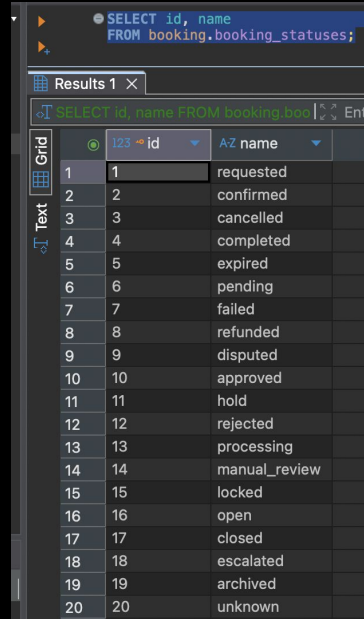
|    | user_id | name    | email        | role  |
|----|---------|---------|--------------|-------|
| 1  | 1       | User 01 | u01@mail.com | guest |
| 2  | 2       | User 02 | u02@mail.com | guest |
| 3  | 3       | User 03 | u03@mail.com | guest |
| 4  | 4       | User 04 | u04@mail.com | guest |
| 5  | 5       | User 05 | u05@mail.com | guest |
| 6  | 6       | User 06 | u06@mail.com | host  |
| 7  | 7       | User 07 | u07@mail.com | host  |
| 8  | 8       | User 08 | u08@mail.com | host  |
| 9  | 9       | User 09 | u09@mail.com | host  |
| 10 | 10      | User 10 | u10@mail.com | host  |
| 11 | 11      | User 11 | u11@mail.com | admin |
| 12 | 12      | User 12 | u12@mail.com | admin |
| 13 | 13      | User 13 | u13@mail.com | guest |
| 14 | 14      | User 14 | u14@mail.com | guest |
| 15 | 15      | User 15 | u15@mail.com | guest |
| 16 | 16      | User 16 | u16@mail.com | guest |
| 17 | 17      | User 17 | u17@mail.com | guest |
| 18 | 18      | User 18 | u18@mail.com | guest |
| 19 | 19      | User 19 | u19@mail.com | guest |
| 20 | 20      | User 20 | u20@mail.com | guest |

# Entity: booking\_statuses

- **Purpose:**  
Lookup table defining possible booking states

```
● CREATE TABLE booking.booking_statuses (  
  id INT NOT NULL,  
  name VARCHAR NOT NULL,  
  PRIMARY KEY (id)  
);
```

- **Test Case**



The screenshot shows a database query interface. At the top, a query is entered: `SELECT id, name FROM booking.booking_statuses;`. Below the query, the results are displayed in a table. The table has two columns: 'id' and 'name'. The 'id' column is highlighted with a blue background. The 'name' column is highlighted with a blue background. The table contains 20 rows of data, numbered 1 to 20. The names are: requested, confirmed, cancelled, completed, expired, pending, failed, refunded, disputed, approved, hold, rejected, processing, manual\_review, locked, open, closed, escalated, archived, and unknown.

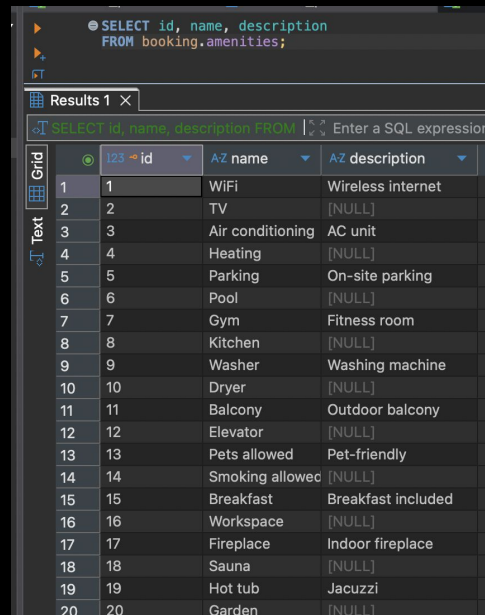
| id | name          |
|----|---------------|
| 1  | requested     |
| 2  | confirmed     |
| 3  | cancelled     |
| 4  | completed     |
| 5  | expired       |
| 6  | pending       |
| 7  | failed        |
| 8  | refunded      |
| 9  | disputed      |
| 10 | approved      |
| 11 | hold          |
| 12 | rejected      |
| 13 | processing    |
| 14 | manual_review |
| 15 | locked        |
| 16 | open          |
| 17 | closed        |
| 18 | escalated     |
| 19 | archived      |
| 20 | unknown       |

# Entity: amenities

- Purpose:  
Lookup table describing available amenities

```
● CREATE TABLE booking.amenities (  
  id INT NOT NULL,  
  name VARCHAR NOT NULL,  
  description VARCHAR,  
  PRIMARY KEY (id)  
);
```

- Test Case



The screenshot shows a database interface with a SQL query editor at the top and a results grid below. The query is: `SELECT id, name, description FROM booking.amenities;`. The results are displayed in a table with columns: `id`, `name`, and `description`. The table contains 20 rows of data, with some cells containing `[NULL]`.

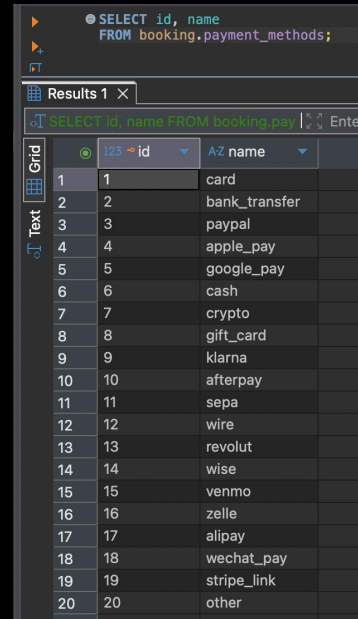
|    | id | name             | description        |
|----|----|------------------|--------------------|
| 1  | 1  | WIFI             | Wireless internet  |
| 2  | 2  | TV               | [NULL]             |
| 3  | 3  | Air conditioning | AC unit            |
| 4  | 4  | Heating          | [NULL]             |
| 5  | 5  | Parking          | On-site parking    |
| 6  | 6  | Pool             | [NULL]             |
| 7  | 7  | Gym              | Fitness room       |
| 8  | 8  | Kitchen          | [NULL]             |
| 9  | 9  | Washer           | Washing machine    |
| 10 | 10 | Dryer            | [NULL]             |
| 11 | 11 | Balcony          | Outdoor balcony    |
| 12 | 12 | Elevator         | [NULL]             |
| 13 | 13 | Pets allowed     | Pet-friendly       |
| 14 | 14 | Smoking allowed  | [NULL]             |
| 15 | 15 | Breakfast        | Breakfast included |
| 16 | 16 | Workspace        | [NULL]             |
| 17 | 17 | Fireplace        | Indoor fireplace   |
| 18 | 18 | Sauna            | [NULL]             |
| 19 | 19 | Hot tub          | Jacuzzi            |
| 20 | 20 | Garden           | [NULL]             |

# Entity: payment\_methods

- **Purpose:**  
Lookup table describing available payment methods

```
● CREATE TABLE booking.payment_methods (  
  id INT NOT NULL,  
  name VARCHAR NOT NULL,  
  PRIMARY KEY (id)  
);
```

- **Test Case**



The screenshot shows a database query result for the query: `SELECT id, name FROM booking.payment_methods;`. The results are displayed in a table with 20 rows. The first column is labeled 'id' and the second column is labeled 'name'. The data is as follows:

| id | name          |
|----|---------------|
| 1  | card          |
| 2  | bank_transfer |
| 3  | paypal        |
| 4  | apple_pay     |
| 5  | google_pay    |
| 6  | cash          |
| 7  | crypto        |
| 8  | gift_card     |
| 9  | klarna        |
| 10 | afterpay      |
| 11 | sepa          |
| 12 | wire          |
| 13 | revolut       |
| 14 | wise          |
| 15 | venmo         |
| 16 | zelle         |
| 17 | alipay        |
| 18 | wechat_pay    |
| 19 | stripe_link   |
| 20 | other         |

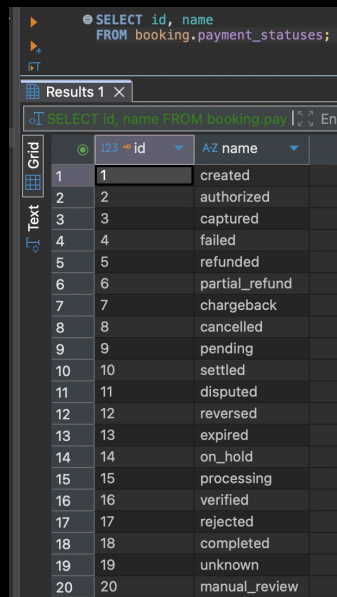


# Entity: payment\_statuses

- **Purpose:**  
Lookup table describing the statuses in which payments can exist

```
● CREATE TABLE booking.payment_statuses (  
  id INT NOT NULL,  
  name VARCHAR NOT NULL,  
  PRIMARY KEY (id)  
);
```

- **Test Case**



The screenshot shows a database query interface. At the top, a SQL query is entered: `SELECT id, name FROM booking.payment_statuses;`. Below the query, the results are displayed in a grid view. The grid has two columns: 'id' and 'name'. The 'id' column is sorted in ascending order, and the 'name' column is sorted alphabetically. The results show 20 rows of data, each with a unique 'id' and a corresponding 'name'.

|    | id | name           |
|----|----|----------------|
| 1  | 1  | created        |
| 2  | 2  | authorized     |
| 3  | 3  | captured       |
| 4  | 4  | failed         |
| 5  | 5  | refunded       |
| 6  | 6  | partial_refund |
| 7  | 7  | chargeback     |
| 8  | 8  | cancelled      |
| 9  | 9  | pending        |
| 10 | 10 | settled        |
| 11 | 11 | disputed       |
| 12 | 12 | reversed       |
| 13 | 13 | expired        |
| 14 | 14 | on_hold        |
| 15 | 15 | processing     |
| 16 | 16 | verified       |
| 17 | 17 | rejected       |
| 18 | 18 | completed      |
| 19 | 19 | unknown        |
| 20 | 20 | manual_review  |

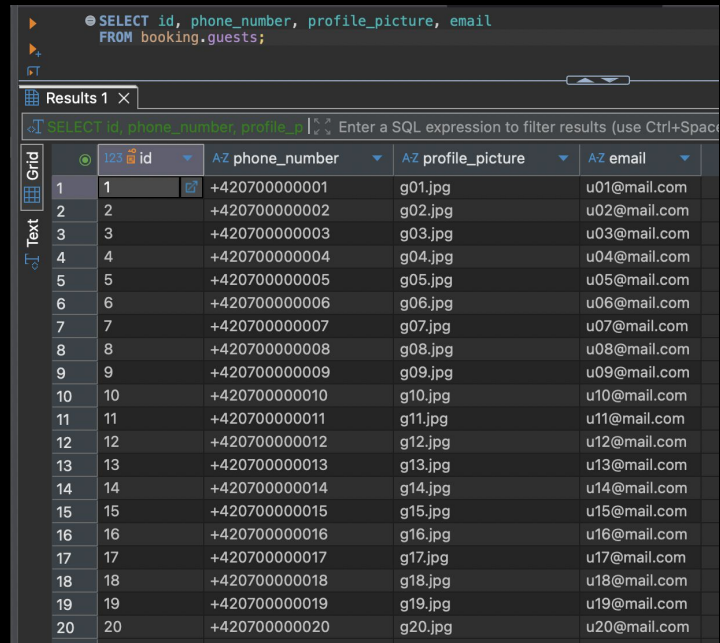
# Entity: guests

- **Purpose:**

Table describing a user type: guest and the information that is available or required on their account. Required: phone number and email

```
● CREATE TABLE booking.guests (  
  id INT NOT NULL,  
  phone_number VARCHAR NOT NULL,  
  profile_picture VARCHAR,  
  email VARCHAR NOT NULL,  
  PRIMARY KEY (id),  
  FOREIGN KEY (id) REFERENCES booking.users(user_id)  
);
```

- **Test Case**



The screenshot shows a database query tool interface. At the top, a SQL query is entered: `SELECT id, phone_number, profile_picture, email FROM booking.guests;`. Below the query, the results are displayed in a grid view. The grid has 5 columns: `id`, `phone_number`, `profile_picture`, and `email`. The results show 20 rows of data, with `id` values from 1 to 20, `phone_number` values starting with +420700000000, `profile_picture` values like g01.jpg, and `email` values like u01@mail.com.

|    | id | phone_number   | profile_picture | email        |
|----|----|----------------|-----------------|--------------|
| 1  | 1  | +4207000000001 | g01.jpg         | u01@mail.com |
| 2  | 2  | +4207000000002 | g02.jpg         | u02@mail.com |
| 3  | 3  | +4207000000003 | g03.jpg         | u03@mail.com |
| 4  | 4  | +4207000000004 | g04.jpg         | u04@mail.com |
| 5  | 5  | +4207000000005 | g05.jpg         | u05@mail.com |
| 6  | 6  | +4207000000006 | g06.jpg         | u06@mail.com |
| 7  | 7  | +4207000000007 | g07.jpg         | u07@mail.com |
| 8  | 8  | +4207000000008 | g08.jpg         | u08@mail.com |
| 9  | 9  | +4207000000009 | g09.jpg         | u09@mail.com |
| 10 | 10 | +4207000000010 | g10.jpg         | u10@mail.com |
| 11 | 11 | +4207000000011 | g11.jpg         | u11@mail.com |
| 12 | 12 | +4207000000012 | g12.jpg         | u12@mail.com |
| 13 | 13 | +4207000000013 | g13.jpg         | u13@mail.com |
| 14 | 14 | +4207000000014 | g14.jpg         | u14@mail.com |
| 15 | 15 | +4207000000015 | g15.jpg         | u15@mail.com |
| 16 | 16 | +4207000000016 | g16.jpg         | u16@mail.com |
| 17 | 17 | +4207000000017 | g17.jpg         | u17@mail.com |
| 18 | 18 | +4207000000018 | g18.jpg         | u18@mail.com |
| 19 | 19 | +4207000000019 | g19.jpg         | u19@mail.com |
| 20 | 20 | +4207000000020 | g20.jpg         | u20@mail.com |

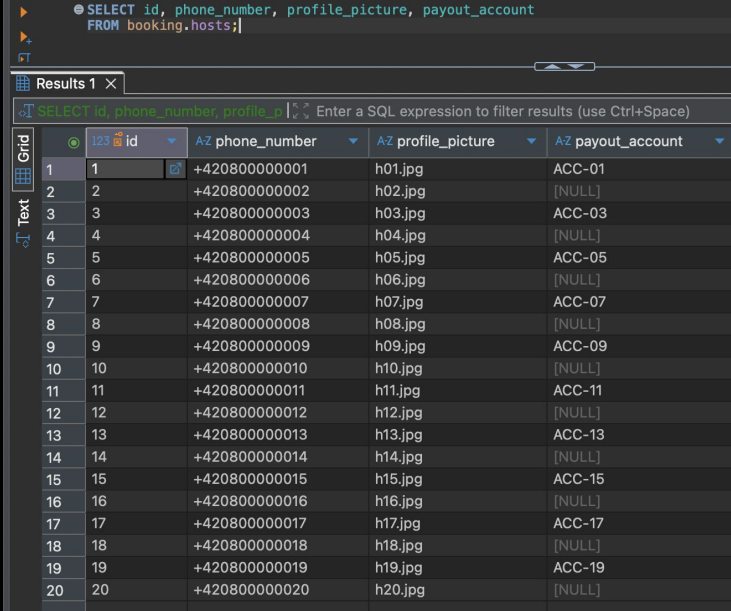
# Entity: hosts

- **Purpose:**

Table describing a user type: host and the information that is available or required on their account. Required: phone number

```
● CREATE TABLE booking.hosts (  
  id INT NOT NULL,  
  phone_number VARCHAR NOT NULL,  
  profile_picture VARCHAR,  
  payout_account VARCHAR,  
  PRIMARY KEY (id),  
  FOREIGN KEY (id) REFERENCES booking.users(user_id)  
);
```

- **Test Case**



The screenshot shows a database management tool interface. At the top, a SQL query is entered: `SELECT id, phone_number, profile_picture, payout_account FROM booking.hosts;`. Below the query, the results are displayed in a table with 4 columns: `id`, `phone_number`, `profile_picture`, and `payout_account`. The results table contains 20 rows of data. The `id` column ranges from 1 to 20. The `phone_number` column contains values like `+420800000001` through `+420800000020`. The `profile_picture` column contains values like `h01.jpg` through `h20.jpg`. The `payout_account` column contains values like `ACC-01` through `ACC-19`, with the last row (id=20) showing `[NULL]`.

|    | id | phone_number  | profile_picture | payout_account |
|----|----|---------------|-----------------|----------------|
| 1  | 1  | +420800000001 | h01.jpg         | ACC-01         |
| 2  | 2  | +420800000002 | h02.jpg         | [NULL]         |
| 3  | 3  | +420800000003 | h03.jpg         | ACC-03         |
| 4  | 4  | +420800000004 | h04.jpg         | [NULL]         |
| 5  | 5  | +420800000005 | h05.jpg         | ACC-05         |
| 6  | 6  | +420800000006 | h06.jpg         | [NULL]         |
| 7  | 7  | +420800000007 | h07.jpg         | ACC-07         |
| 8  | 8  | +420800000008 | h08.jpg         | [NULL]         |
| 9  | 9  | +420800000009 | h09.jpg         | ACC-09         |
| 10 | 10 | +420800000010 | h10.jpg         | [NULL]         |
| 11 | 11 | +420800000011 | h11.jpg         | ACC-11         |
| 12 | 12 | +420800000012 | h12.jpg         | [NULL]         |
| 13 | 13 | +420800000013 | h13.jpg         | ACC-13         |
| 14 | 14 | +420800000014 | h14.jpg         | [NULL]         |
| 15 | 15 | +420800000015 | h15.jpg         | ACC-15         |
| 16 | 16 | +420800000016 | h16.jpg         | [NULL]         |
| 17 | 17 | +420800000017 | h17.jpg         | ACC-17         |
| 18 | 18 | +420800000018 | h18.jpg         | [NULL]         |
| 19 | 19 | +420800000019 | h19.jpg         | ACC-19         |
| 20 | 20 | +420800000020 | h20.jpg         | [NULL]         |

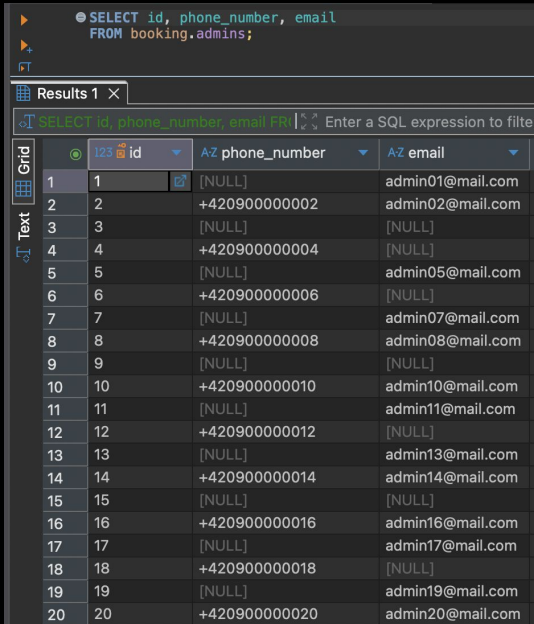
# Entity: admins

- **Purpose:**

Table describing a user type: admin and the information that is available or required on their account. Required: phone number

```
● CREATE TABLE booking.admins (  
  id INT NOT NULL,  
  phone_number VARCHAR,  
  email VARCHAR,  
  PRIMARY KEY (id),  
  FOREIGN KEY (id) REFERENCES booking.users(user_id)  
);
```

- **Test Case**



● SELECT id, phone\_number, email  
FROM booking.admins;

Results 1 X

SELECT id, phone\_number, email FROM Enter a SQL expression to filter

|    | id | phone_number  | email            |
|----|----|---------------|------------------|
| 1  | 1  | [NULL]        | admin01@mail.com |
| 2  | 2  | +420900000002 | admin02@mail.com |
| 3  | 3  | [NULL]        | [NULL]           |
| 4  | 4  | +420900000004 | [NULL]           |
| 5  | 5  | [NULL]        | admin05@mail.com |
| 6  | 6  | +420900000006 | [NULL]           |
| 7  | 7  | [NULL]        | admin07@mail.com |
| 8  | 8  | +420900000008 | admin08@mail.com |
| 9  | 9  | [NULL]        | [NULL]           |
| 10 | 10 | +420900000010 | admin10@mail.com |
| 11 | 11 | [NULL]        | admin11@mail.com |
| 12 | 12 | +420900000012 | [NULL]           |
| 13 | 13 | [NULL]        | admin13@mail.com |
| 14 | 14 | +420900000014 | admin14@mail.com |
| 15 | 15 | [NULL]        | [NULL]           |
| 16 | 16 | +420900000016 | admin16@mail.com |
| 17 | 17 | [NULL]        | admin17@mail.com |
| 18 | 18 | +420900000018 | [NULL]           |
| 19 | 19 | [NULL]        | admin19@mail.com |
| 20 | 20 | +420900000020 | admin20@mail.com |

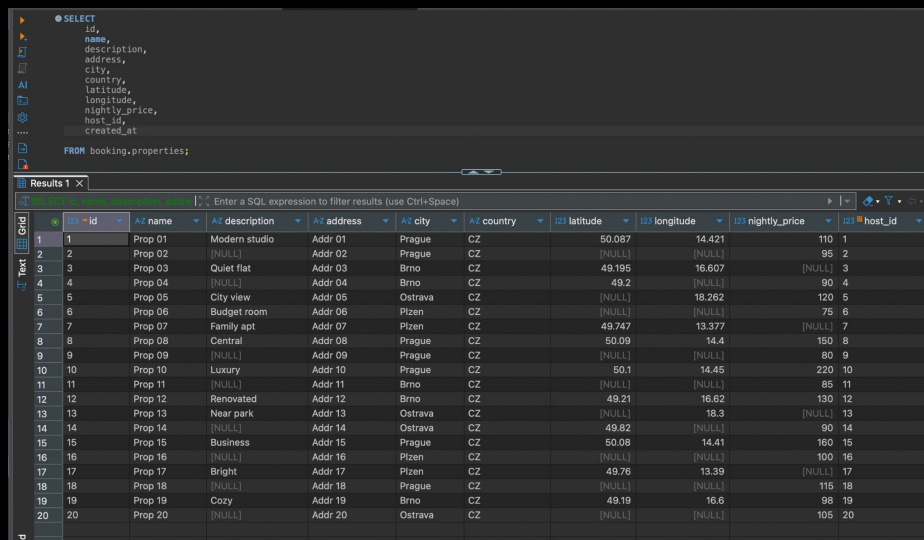
# Entity: properties

- Purpose:

Table storing key data about rental properties, their location, description and related host.

```
CREATE TABLE booking.properties (  
  id INT NOT NULL,  
  name VARCHAR,  
  description VARCHAR,  
  address VARCHAR,  
  city VARCHAR,  
  country VARCHAR,  
  latitude NUMERIC,  
  longitude NUMERIC,  
  nightly_price NUMERIC,  
  host_id INT NOT NULL,  
  created_at TIMESTAMP,  
  PRIMARY KEY (id),  
  FOREIGN KEY (host_id) REFERENCES booking.hosts(id)  
);
```

- Test Case



The screenshot shows a database client interface. At the top, a SQL query is entered in a text area:

```
SELECT  
  id,  
  name,  
  description,  
  address,  
  city,  
  country,  
  latitude,  
  longitude,  
  nightly_price,  
  host_id,  
  created_at  
FROM booking.properties;
```

Below the query, the results are displayed in a table grid. The table has 15 columns: id, name, description, address, city, country, latitude, longitude, nightly\_price, and host\_id. The data is as follows:

|    | id | name    | description   | address | city    | country | latitude | longitude | nightly_price | host_id |
|----|----|---------|---------------|---------|---------|---------|----------|-----------|---------------|---------|
| 1  | 1  | Prop 01 | Modern studio | Addr 01 | Prague  | CZ      | 50.067   | 14.421    | 110           | 1       |
| 2  | 2  | Prop 02 | [NULL]        | Addr 02 | Prague  | CZ      | [NULL]   | [NULL]    | 95            | 2       |
| 3  | 3  | Prop 03 | Quiet flat    | Addr 03 | Brno    | CZ      | 49.195   | 16.607    | [NULL]        | 3       |
| 4  | 4  | Prop 04 | [NULL]        | Addr 04 | Brno    | CZ      | 49.2     | [NULL]    | 90            | 4       |
| 5  | 5  | Prop 05 | City view     | Addr 05 | Ostrava | CZ      | [NULL]   | 18.262    | 120           | 5       |
| 6  | 6  | Prop 06 | Budget room   | Addr 06 | Pízen   | CZ      | [NULL]   | [NULL]    | 75            | 6       |
| 7  | 7  | Prop 07 | Family apt    | Addr 07 | Pízen   | CZ      | 49.747   | 13.377    | [NULL]        | 7       |
| 8  | 8  | Prop 08 | Central       | Addr 08 | Prague  | CZ      | 50.09    | 14.4      | 150           | 8       |
| 9  | 9  | Prop 09 | [NULL]        | Addr 09 | Prague  | CZ      | [NULL]   | [NULL]    | 80            | 9       |
| 10 | 10 | Prop 10 | Luxury        | Addr 10 | Prague  | CZ      | 50.1     | 14.45     | 220           | 10      |
| 11 | 11 | Prop 11 | [NULL]        | Addr 11 | Brno    | CZ      | [NULL]   | [NULL]    | 85            | 11      |
| 12 | 12 | Prop 12 | Renovated     | Addr 12 | Brno    | CZ      | 49.21    | 16.62     | 130           | 12      |
| 13 | 13 | Prop 13 | Near park     | Addr 13 | Ostrava | CZ      | [NULL]   | 18.3      | [NULL]        | 13      |
| 14 | 14 | Prop 14 | [NULL]        | Addr 14 | Ostrava | CZ      | 49.82    | [NULL]    | 90            | 14      |
| 15 | 15 | Prop 15 | Business      | Addr 15 | Prague  | CZ      | 50.08    | 14.41     | 160           | 15      |
| 16 | 16 | Prop 16 | [NULL]        | Addr 16 | Pízen   | CZ      | [NULL]   | [NULL]    | 100           | 16      |
| 17 | 17 | Prop 17 | Bright        | Addr 17 | Pízen   | CZ      | 49.76    | 13.39     | [NULL]        | 17      |
| 18 | 18 | Prop 18 | [NULL]        | Addr 18 | Prague  | CZ      | [NULL]   | [NULL]    | 115           | 18      |
| 19 | 19 | Prop 19 | Cozy          | Addr 19 | Brno    | CZ      | 49.19    | 16.6      | 98            | 19      |
| 20 | 20 | Prop 20 | [NULL]        | Addr 20 | Ostrava | CZ      | [NULL]   | [NULL]    | 105           | 20      |

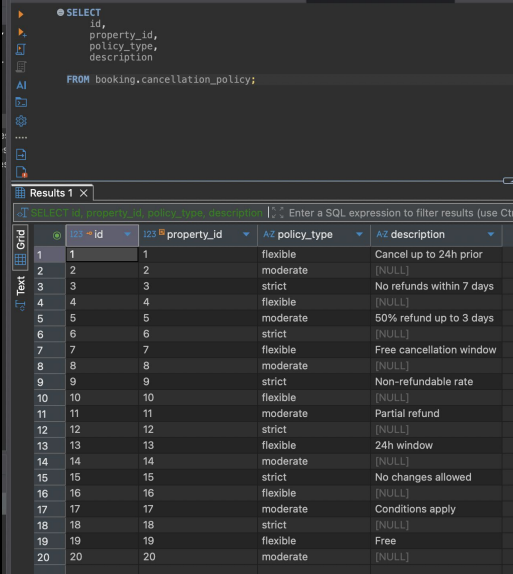
# Entity: cancellation\_policy

- Purpose:

Table storing key data about each property's cancellation policy..

```
● CREATE TABLE booking.cancellation_policy (  
  id INT NOT NULL,  
  property_id INT NOT NULL,  
  policy_type VARCHAR,  
  description VARCHAR,  
  PRIMARY KEY (id),  
  FOREIGN KEY (property_id) REFERENCES booking.properties(id)  
);
```

- Test Case



The screenshot shows a SQL query editor with the following query:

```
● SELECT  
  id,  
  property_id,  
  policy_type,  
  description  
FROM booking.cancellation_policy;
```

Below the query, the results are displayed in a table grid. The table has 4 columns: id, property\_id, policy\_type, and description. The results show 20 rows of data.

|    | id | property_id | policy_type | description              |
|----|----|-------------|-------------|--------------------------|
| 1  | 1  | 1           | flexible    | Cancel up to 24h prior   |
| 2  | 2  | 2           | moderate    | [NULL]                   |
| 3  | 3  | 3           | strict      | No refunds within 7 days |
| 4  | 4  | 4           | flexible    | [NULL]                   |
| 5  | 5  | 5           | moderate    | 50% refund up to 3 days  |
| 6  | 6  | 6           | strict      | [NULL]                   |
| 7  | 7  | 7           | flexible    | Free cancellation window |
| 8  | 8  | 8           | moderate    | [NULL]                   |
| 9  | 9  | 9           | strict      | Non-refundable rate      |
| 10 | 10 | 10          | flexible    | [NULL]                   |
| 11 | 11 | 11          | moderate    | Partial refund           |
| 12 | 12 | 12          | strict      | [NULL]                   |
| 13 | 13 | 13          | flexible    | 24h window               |
| 14 | 14 | 14          | moderate    | [NULL]                   |
| 15 | 15 | 15          | strict      | No changes allowed       |
| 16 | 16 | 16          | flexible    | [NULL]                   |
| 17 | 17 | 17          | moderate    | Conditions apply         |
| 18 | 18 | 18          | strict      | [NULL]                   |
| 19 | 19 | 19          | flexible    | Free                     |
| 20 | 20 | 20          | moderate    | [NULL]                   |

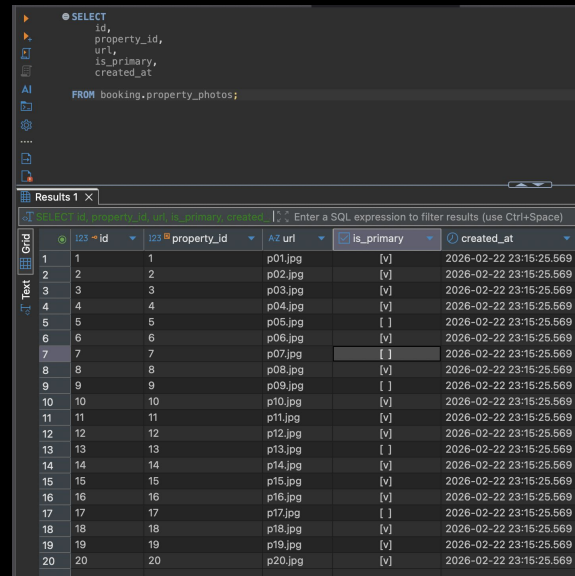
# Entity: property\_photos

- **Purpose:**

Table storing the reference to the photos of each property, their respective URL and whether they're primary

```
● CREATE TABLE booking.property_photos (  
  id INT NOT NULL,  
  property_id INT NOT NULL,  
  url VARCHAR NOT NULL,  
  is_primary BOOLEAN,  
  created_at TIMESTAMPT,  
  PRIMARY KEY (id),  
  FOREIGN KEY (property_id) REFERENCES booking.properties(id)  
);
```

- **Test Case**



The screenshot shows a database client interface. The top pane displays a SQL query: `SELECT id, property_id, url, is_primary, created_at FROM booking.property_photos;`. The bottom pane shows the results of this query in a table format. The table has 5 columns: `id`, `property_id`, `url`, `is_primary`, and `created_at`. The results show 20 rows of data, with `id` ranging from 1 to 20, `property_id` ranging from 1 to 20, `url` values like `p01.jpg` to `p20.jpg`, `is_primary` values like `[v]` or `[ ]`, and `created_at` timestamps.

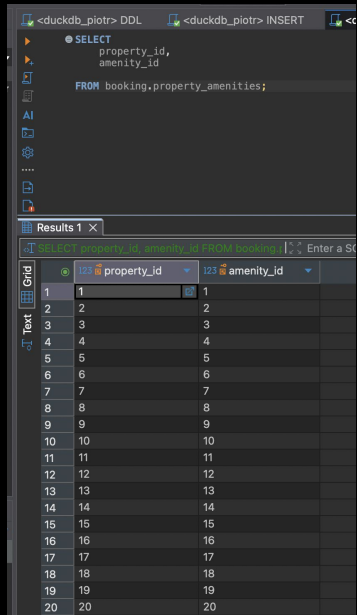
|    | id | property_id | url     | is_primary | created_at              |
|----|----|-------------|---------|------------|-------------------------|
| 1  | 1  | 1           | p01.jpg | [v]        | 2026-02-22 23:15:25.569 |
| 2  | 2  | 2           | p02.jpg | [v]        | 2026-02-22 23:15:25.569 |
| 3  | 3  | 3           | p03.jpg | [v]        | 2026-02-22 23:15:25.569 |
| 4  | 4  | 4           | p04.jpg | [v]        | 2026-02-22 23:15:25.569 |
| 5  | 5  | 5           | p05.jpg | [ ]        | 2026-02-22 23:15:25.569 |
| 6  | 6  | 6           | p06.jpg | [v]        | 2026-02-22 23:15:25.569 |
| 7  | 7  | 7           | p07.jpg | [ ]        | 2026-02-22 23:15:25.569 |
| 8  | 8  | 8           | p08.jpg | [v]        | 2026-02-22 23:15:25.569 |
| 9  | 9  | 9           | p09.jpg | [ ]        | 2026-02-22 23:15:25.569 |
| 10 | 10 | 10          | p10.jpg | [v]        | 2026-02-22 23:15:25.569 |
| 11 | 11 | 11          | p11.jpg | [v]        | 2026-02-22 23:15:25.569 |
| 12 | 12 | 12          | p12.jpg | [v]        | 2026-02-22 23:15:25.569 |
| 13 | 13 | 13          | p13.jpg | [ ]        | 2026-02-22 23:15:25.569 |
| 14 | 14 | 14          | p14.jpg | [v]        | 2026-02-22 23:15:25.569 |
| 15 | 15 | 15          | p15.jpg | [v]        | 2026-02-22 23:15:25.569 |
| 16 | 16 | 16          | p16.jpg | [v]        | 2026-02-22 23:15:25.569 |
| 17 | 17 | 17          | p17.jpg | [ ]        | 2026-02-22 23:15:25.569 |
| 18 | 18 | 18          | p18.jpg | [v]        | 2026-02-22 23:15:25.569 |
| 19 | 19 | 19          | p19.jpg | [v]        | 2026-02-22 23:15:25.569 |
| 20 | 20 | 20          | p20.jpg | [v]        | 2026-02-22 23:15:25.569 |

# Entity: property\_amenities

- **Purpose:**  
Table linking properties to amenities.

```
● CREATE TABLE booking.property_amenities (  
  property_id INT NOT NULL,  
  amenity_id INT NOT NULL,  
  PRIMARY KEY (property_id, amenity_id),  
  FOREIGN KEY (property_id) REFERENCES booking.properties(id),  
  FOREIGN KEY (amenity_id) REFERENCES booking.amenities(id)  
);
```

- **Test Case**



The screenshot shows a database client interface with a query editor and a results pane. The query editor contains a SELECT statement: `SELECT property_id, amenity_id FROM booking.property_amenities;`. The results pane shows a table with two columns: `property_id` and `amenity_id`. The table contains 20 rows of data, where each row has the same value in both columns, ranging from 1 to 20.

|    | property_id | amenity_id |
|----|-------------|------------|
| 1  | 1           | 1          |
| 2  | 2           | 2          |
| 3  | 3           | 3          |
| 4  | 4           | 4          |
| 5  | 5           | 5          |
| 6  | 6           | 6          |
| 7  | 7           | 7          |
| 8  | 8           | 8          |
| 9  | 9           | 9          |
| 10 | 10          | 10         |
| 11 | 11          | 11         |
| 12 | 12          | 12         |
| 13 | 13          | 13         |
| 14 | 14          | 14         |
| 15 | 15          | 15         |
| 16 | 16          | 16         |
| 17 | 17          | 17         |
| 18 | 18          | 18         |
| 19 | 19          | 19         |
| 20 | 20          | 20         |



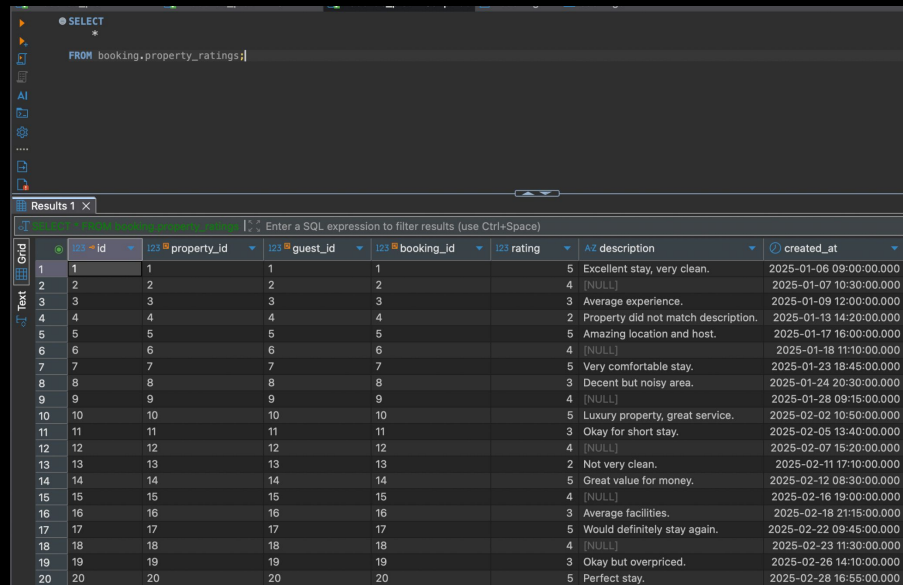
# Entity: property\_ratings

- Purpose:

Table linking properties and bookings to ratings.

```
● CREATE TABLE booking.property_ratings (  
  id INT,  
  property_id INT NOT NULL,  
  guest_id INT NOT NULL,  
  booking_id INT NOT NULL,  
  rating INT NOT NULL,  
  description VARCHAR,  
  created_at TIMESTAMPTZ,  
  PRIMARY KEY (id),  
  FOREIGN KEY (property_id) REFERENCES booking.properties(id),  
  FOREIGN KEY (guest_id) REFERENCES booking.guests(id),  
  FOREIGN KEY (booking_id) REFERENCES booking.bookings(id)  
);
```

- Test Case



The screenshot shows a database query editor with a SELECT statement and its results. The query is:

```
SELECT  
FROM booking.property_ratings;
```

The results are displayed in a table with 20 rows. The columns are: id, property\_id, guest\_id, booking\_id, rating, description, and created\_at.

|    | id | property_id | guest_id | booking_id | rating | description                         | created_at              |
|----|----|-------------|----------|------------|--------|-------------------------------------|-------------------------|
| 1  | 1  | 1           | 1        | 1          | 5      | Excellent stay, very clean.         | 2025-01-06 09:00:00.000 |
| 2  | 2  | 2           | 2        | 2          | 4      | [NULL]                              | 2025-01-07 10:30:00.000 |
| 3  | 3  | 3           | 3        | 3          | 3      | Average experience.                 | 2025-01-09 12:00:00.000 |
| 4  | 4  | 4           | 4        | 4          | 2      | Property did not match description. | 2025-01-13 14:20:00.000 |
| 5  | 5  | 5           | 5        | 5          | 5      | Amazing location and host.          | 2025-01-17 16:00:00.000 |
| 6  | 6  | 6           | 6        | 6          | 4      | [NULL]                              | 2025-01-18 11:10:00.000 |
| 7  | 7  | 7           | 7        | 7          | 5      | Very comfortable stay.              | 2025-01-23 18:45:00.000 |
| 8  | 8  | 8           | 8        | 8          | 3      | Decent but noisy area.              | 2025-01-24 20:30:00.000 |
| 9  | 9  | 9           | 9        | 9          | 4      | [NULL]                              | 2025-01-28 09:15:00.000 |
| 10 | 10 | 10          | 10       | 10         | 5      | Luxury property, great service.     | 2025-02-02 10:50:00.000 |
| 11 | 11 | 11          | 11       | 11         | 3      | Okay for short stay.                | 2025-02-05 13:40:00.000 |
| 12 | 12 | 12          | 12       | 12         | 4      | [NULL]                              | 2025-02-07 15:20:00.000 |
| 13 | 13 | 13          | 13       | 13         | 2      | Not very clean.                     | 2025-02-11 17:10:00.000 |
| 14 | 14 | 14          | 14       | 14         | 5      | Great value for money.              | 2025-02-12 08:30:00.000 |
| 15 | 15 | 15          | 15       | 15         | 4      | [NULL]                              | 2025-02-16 19:00:00.000 |
| 16 | 16 | 16          | 16       | 16         | 3      | Average facilities.                 | 2025-02-18 21:15:00.000 |
| 17 | 17 | 17          | 17       | 17         | 5      | Would definitely stay again.        | 2025-02-22 09:45:00.000 |
| 18 | 18 | 18          | 18       | 18         | 4      | [NULL]                              | 2025-02-23 11:30:00.000 |
| 19 | 19 | 19          | 19       | 19         | 3      | Okay but overpriced.                | 2025-02-26 14:10:00.000 |
| 20 | 20 | 20          | 20       | 20         | 5      | Perfect stay.                       | 2025-02-28 16:55:00.000 |

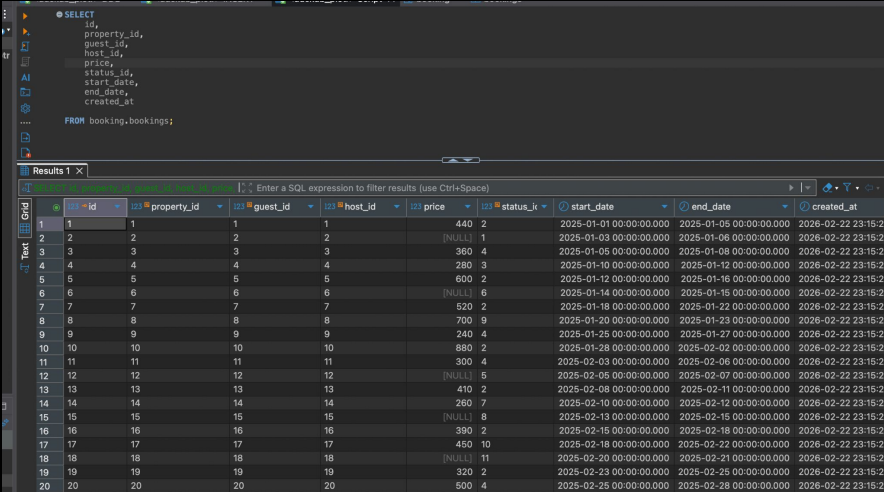
# Entity: bookings

- **Purpose:**

Key table working as the main source of booking related data, including the host, guest, linked property, time periods and price

```
● CREATE TABLE booking.bookings (  
  id INT NOT NULL,  
  property_id INT NOT NULL,  
  guest_id INT NOT NULL,  
  host_id INT NOT NULL,  
  price NUMERIC,  
  status_id INT NOT NULL,  
  start_date DATE,  
  end_date DATE,  
  created_at TIMESTAMP,  
  PRIMARY KEY (id),  
  FOREIGN KEY (guest_id) REFERENCES booking.guests(id),  
  FOREIGN KEY (host_id) REFERENCES booking.hosts(id),  
  FOREIGN KEY (property_id) REFERENCES booking.properties(id),  
  FOREIGN KEY (status_id) REFERENCES booking.booking_statuses(id)  
);
```

- **Test Case**



The screenshot shows a database query interface. At the top, a SQL query is entered: `SELECT id, property_id, guest_id, host_id, price, status_id, start_date, end_date, created_at FROM booking.bookings;`. Below the query, the results are displayed in a table with 9 columns: `id`, `property_id`, `guest_id`, `host_id`, `price`, `status_k`, `start_date`, `end_date`, and `created_at`. The table contains 20 rows of data, with some cells containing NULL values.

|    | id | property_id | guest_id | host_id | price  | status_k | start_date              | end_date                | created_at         |
|----|----|-------------|----------|---------|--------|----------|-------------------------|-------------------------|--------------------|
| 1  | 1  | 1           | 1        | 1       | 440    | 2        | 2025-01-01 00:00:00.000 | 2025-01-05 00:00:00.000 | 2026-02-22 23:15:2 |
| 2  | 2  | 2           | 2        | 2       | [NULL] | 1        | 2025-01-03 00:00:00.000 | 2025-01-06 00:00:00.000 | 2026-02-22 23:15:2 |
| 3  | 3  | 3           | 3        | 3       | 360    | 4        | 2025-01-05 00:00:00.000 | 2025-01-08 00:00:00.000 | 2026-02-22 23:15:2 |
| 4  | 4  | 4           | 4        | 4       | 280    | 3        | 2025-01-10 00:00:00.000 | 2025-01-13 00:00:00.000 | 2026-02-22 23:15:2 |
| 5  | 5  | 5           | 5        | 5       | 600    | 2        | 2025-01-12 00:00:00.000 | 2025-01-16 00:00:00.000 | 2026-02-22 23:15:2 |
| 6  | 6  | 6           | 6        | 6       | [NULL] | 6        | 2025-01-14 00:00:00.000 | 2025-01-16 00:00:00.000 | 2026-02-22 23:15:2 |
| 7  | 7  | 7           | 7        | 7       | 520    | 2        | 2025-01-18 00:00:00.000 | 2025-01-22 00:00:00.000 | 2026-02-22 23:15:2 |
| 8  | 8  | 8           | 8        | 8       | 700    | 9        | 2025-01-20 00:00:00.000 | 2025-01-23 00:00:00.000 | 2026-02-22 23:15:2 |
| 9  | 9  | 9           | 9        | 9       | 240    | 4        | 2025-01-25 00:00:00.000 | 2025-02-27 00:00:00.000 | 2026-02-22 23:15:2 |
| 10 | 10 | 10          | 10       | 10      | 880    | 2        | 2025-01-28 00:00:00.000 | 2025-02-02 00:00:00.000 | 2026-02-22 23:15:2 |
| 11 | 11 | 11          | 11       | 11      | 300    | 4        | 2025-02-03 00:00:00.000 | 2025-02-06 00:00:00.000 | 2026-02-22 23:15:2 |
| 12 | 12 | 12          | 12       | 12      | [NULL] | 5        | 2025-02-05 00:00:00.000 | 2025-02-07 00:00:00.000 | 2026-02-22 23:15:2 |
| 13 | 13 | 13          | 13       | 13      | 410    | 2        | 2025-02-08 00:00:00.000 | 2025-02-11 00:00:00.000 | 2026-02-22 23:15:2 |
| 14 | 14 | 14          | 14       | 14      | 260    | 7        | 2025-02-10 00:00:00.000 | 2025-02-12 00:00:00.000 | 2026-02-22 23:15:2 |
| 15 | 15 | 15          | 15       | 15      | [NULL] | 8        | 2025-02-13 00:00:00.000 | 2025-02-15 00:00:00.000 | 2026-02-22 23:15:2 |
| 16 | 16 | 16          | 16       | 16      | 390    | 2        | 2025-02-15 00:00:00.000 | 2025-02-18 00:00:00.000 | 2026-02-22 23:15:2 |
| 17 | 17 | 17          | 17       | 17      | 450    | 10       | 2025-02-18 00:00:00.000 | 2025-02-22 00:00:00.000 | 2026-02-22 23:15:2 |
| 18 | 18 | 18          | 18       | 18      | [NULL] | 11       | 2025-02-20 00:00:00.000 | 2025-02-21 00:00:00.000 | 2026-02-22 23:15:2 |
| 19 | 19 | 19          | 19       | 19      | 320    | 2        | 2025-02-23 00:00:00.000 | 2025-02-25 00:00:00.000 | 2026-02-22 23:15:2 |
| 20 | 20 | 20          | 20       | 20      | 500    | 4        | 2025-02-25 00:00:00.000 | 2025-02-28 00:00:00.000 | 2026-02-22 23:15:2 |

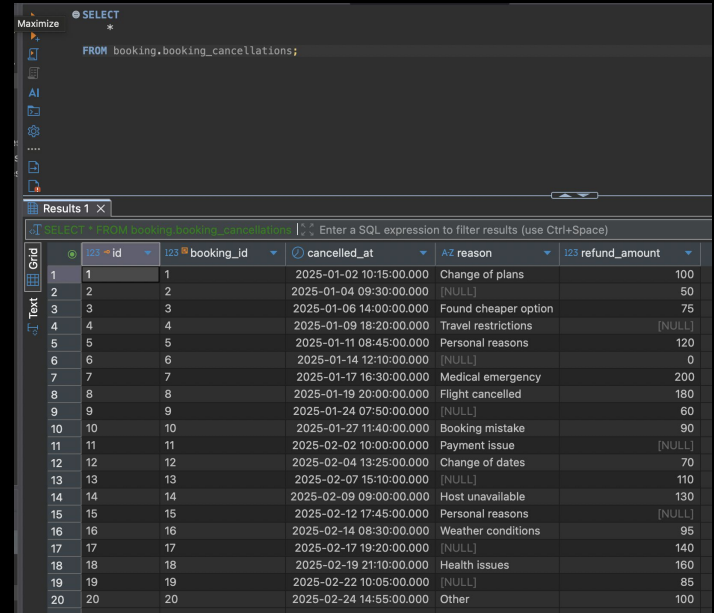
# Entity: booking\_cancellations

- **Purpose:**

Specialized table with data on reasons related to booking cancellations

```
CREATE TABLE booking.booking_cancellations (  
  id INT,  
  booking_id INT NOT NULL,  
  cancelled_at TIMESTAMP NOT NULL,  
  reason VARCHAR,  
  refund_amount NUMERIC,  
  PRIMARY KEY (id),  
  FOREIGN KEY(booking_id) REFERENCES booking.bookings(id)  
);
```

- **Test Case**



The screenshot shows a database interface with a SQL query editor at the top and a results table below. The query is: `SELECT * FROM booking.booking_cancellations;`. The results table has 20 rows and 6 columns: `id`, `booking_id`, `cancelled_at`, `reason`, and `refund_amount`. The data represents various booking cancellations with reasons like 'Change of plans', 'Travel restrictions', 'Medical emergency', etc.

|    | id | booking_id | cancelled_at            | reason               | refund_amount |
|----|----|------------|-------------------------|----------------------|---------------|
| 1  | 1  | 1          | 2025-01-02 10:15:00.000 | Change of plans      | 100           |
| 2  | 2  | 2          | 2025-01-04 09:30:00.000 | [NULL]               | 50            |
| 3  | 3  | 3          | 2025-01-06 14:00:00.000 | Found cheaper option | 75            |
| 4  | 4  | 4          | 2025-01-09 18:20:00.000 | Travel restrictions  | [NULL]        |
| 5  | 5  | 5          | 2025-01-11 08:45:00.000 | Personal reasons     | 120           |
| 6  | 6  | 6          | 2025-01-14 12:10:00.000 | [NULL]               | 0             |
| 7  | 7  | 7          | 2025-01-17 16:30:00.000 | Medical emergency    | 200           |
| 8  | 8  | 8          | 2025-01-19 20:00:00.000 | Flight cancelled     | 180           |
| 9  | 9  | 9          | 2025-01-24 07:50:00.000 | [NULL]               | 60            |
| 10 | 10 | 10         | 2025-01-27 11:40:00.000 | Booking mistake      | 90            |
| 11 | 11 | 11         | 2025-02-02 10:00:00.000 | Payment issue        | [NULL]        |
| 12 | 12 | 12         | 2025-02-04 13:25:00.000 | Change of dates      | 70            |
| 13 | 13 | 13         | 2025-02-07 15:10:00.000 | [NULL]               | 110           |
| 14 | 14 | 14         | 2025-02-09 09:00:00.000 | Host unavailable     | 130           |
| 15 | 15 | 15         | 2025-02-12 17:45:00.000 | Personal reasons     | [NULL]        |
| 16 | 16 | 16         | 2025-02-14 08:30:00.000 | Weather conditions   | 95            |
| 17 | 17 | 17         | 2025-02-17 19:20:00.000 | [NULL]               | 140           |
| 18 | 18 | 18         | 2025-02-19 21:10:00.000 | Health issues        | 160           |
| 19 | 19 | 19         | 2025-02-22 10:05:00.000 | [NULL]               | 85            |
| 20 | 20 | 20         | 2025-02-24 14:55:00.000 | Other                | 100           |

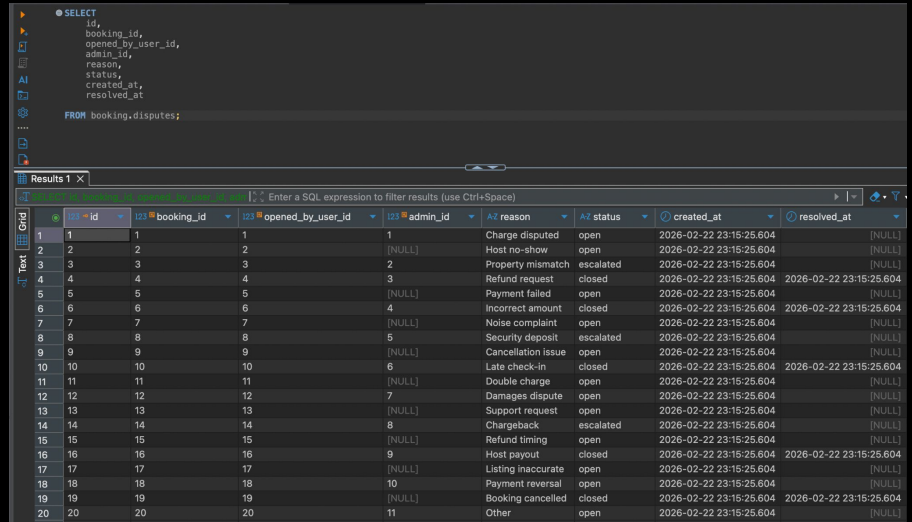
# Entity: disputes

- **Purpose:**

Disputes related to user requested customer support disputes and their details.

```
● CREATE TABLE booking.disputes (  
  id INT NOT NULL,  
  booking_id INT NOT NULL,  
  opened_by_user_id INT NOT NULL,  
  admin_id INT,  
  reason VARCHAR NOT NULL,  
  status VARCHAR,  
  created_at TIMESTAMPT,  
  resolved_at TIMESTAMPT,  
  PRIMARY KEY (id),  
  FOREIGN KEY (booking_id) REFERENCES booking.bookings(id),  
  FOREIGN KEY (opened_by_user_id) REFERENCES booking.users(user_id),  
  FOREIGN KEY (admin_id) REFERENCES booking.admins(id)  
);
```

- **Test Case**



The screenshot shows a SQL query editor with a SELECT statement and its results in a table grid. The query is:

```
SELECT  
  id,  
  booking_id,  
  opened_by_user_id,  
  admin_id,  
  reason,  
  status,  
  created_at,  
  resolved_at  
FROM booking.disputes;
```

The results are displayed in a table grid with the following columns: id, booking\_id, opened\_by\_user\_id, admin\_id, reason, status, created\_at, and resolved\_at. The data is as follows:

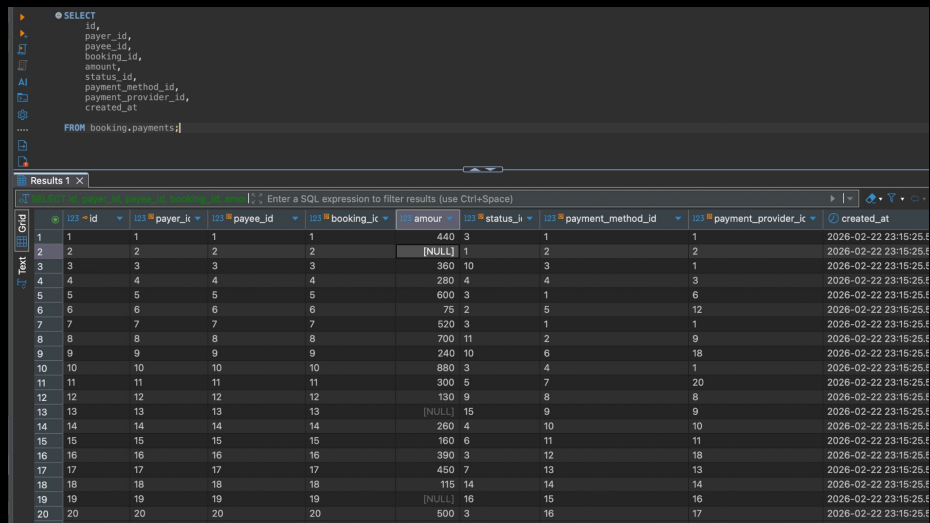
|    | id | booking_id | opened_by_user_id | admin_id | reason             | status    | created_at              | resolved_at             |
|----|----|------------|-------------------|----------|--------------------|-----------|-------------------------|-------------------------|
| 1  | 1  | 1          | 1                 | 1        | Charge disputed    | open      | 2026-02-22 23:15:25.604 | [NULL]                  |
| 2  | 2  | 2          | 2                 | [NULL]   | Host no-show       | open      | 2026-02-22 23:15:25.604 | [NULL]                  |
| 3  | 3  | 3          | 3                 | 2        | Property mismatch  | escalated | 2026-02-22 23:15:25.604 | [NULL]                  |
| 4  | 4  | 4          | 4                 | 3        | Refund request     | closed    | 2026-02-22 23:15:25.604 | 2026-02-22 23:15:25.604 |
| 5  | 5  | 5          | 5                 | [NULL]   | Payment failed     | open      | 2026-02-22 23:15:25.604 | [NULL]                  |
| 6  | 6  | 6          | 6                 | 4        | Incorrect amount   | closed    | 2026-02-22 23:15:25.604 | 2026-02-22 23:15:25.604 |
| 7  | 7  | 7          | 7                 | [NULL]   | Noise complaint    | open      | 2026-02-22 23:15:25.604 | [NULL]                  |
| 8  | 8  | 8          | 8                 | 5        | Security deposit   | escalated | 2026-02-22 23:15:25.604 | [NULL]                  |
| 9  | 9  | 9          | 9                 | [NULL]   | Cancellation issue | open      | 2026-02-22 23:15:25.604 | [NULL]                  |
| 10 | 10 | 10         | 10                | 6        | Late check-in      | closed    | 2026-02-22 23:15:25.604 | 2026-02-22 23:15:25.604 |
| 11 | 11 | 11         | 11                | [NULL]   | Double charge      | open      | 2026-02-22 23:15:25.604 | [NULL]                  |
| 12 | 12 | 12         | 12                | 7        | Damages dispute    | open      | 2026-02-22 23:15:25.604 | [NULL]                  |
| 13 | 13 | 13         | 13                | [NULL]   | Support request    | open      | 2026-02-22 23:15:25.604 | [NULL]                  |
| 14 | 14 | 14         | 14                | 8        | Chargeback         | escalated | 2026-02-22 23:15:25.604 | [NULL]                  |
| 15 | 15 | 15         | 15                | [NULL]   | Refund timing      | open      | 2026-02-22 23:15:25.604 | [NULL]                  |
| 16 | 16 | 16         | 16                | 9        | Host payout        | closed    | 2026-02-22 23:15:25.604 | 2026-02-22 23:15:25.604 |
| 17 | 17 | 17         | 17                | [NULL]   | Listing inaccurate | open      | 2026-02-22 23:15:25.604 | [NULL]                  |
| 18 | 18 | 18         | 18                | 10       | Payment reversal   | open      | 2026-02-22 23:15:25.604 | [NULL]                  |
| 19 | 19 | 19         | 19                | [NULL]   | Booking cancelled  | closed    | 2026-02-22 23:15:25.604 | 2026-02-22 23:15:25.604 |
| 20 | 20 | 20         | 20                | 11       | Other              | open      | 2026-02-22 23:15:25.604 | [NULL]                  |

# Entity: payments

- **Purpose:**  
Key table containing all payment data and related details.

```
● CREATE TABLE booking.payments (  
  id INT NOT NULL,  
  payer_id INT NOT NULL,  
  payee_id INT NOT NULL,  
  booking_id INT NOT NULL,  
  amount NUMERIC,  
  status_id INT NOT NULL,  
  payment_method_id INT NOT NULL,  
  payment_provider_id INT NOT NULL,  
  created_at TIMESTAMP,  
  PRIMARY KEY (id),  
  FOREIGN KEY (payer_id) REFERENCES booking.guests(id),  
  FOREIGN KEY (payee_id) REFERENCES booking.hosts(id),  
  FOREIGN KEY (booking_id) REFERENCES booking.bookings(id),  
  FOREIGN KEY (payment_method_id) REFERENCES booking.payment_methods(id),  
  FOREIGN KEY (status_id) REFERENCES booking.payment_statuses(id),  
  FOREIGN KEY (payment_provider_id) REFERENCES booking.payment_providers(id)  
);
```

- **Test Case**



The screenshot shows a database query interface. At the top, a SQL query is entered in a text area:

```
SELECT  
  id,  
  payer_id,  
  payee_id,  
  booking_id,  
  amount,  
  status_id,  
  payment_method_id,  
  payment_provider_id,  
  created_at  
FROM booking.payments;
```

Below the query, the results are displayed in a table titled "Results 1 X". The table has 11 columns: id, payer\_id, payee\_id, booking\_id, amount, status\_id, payment\_method\_id, payment\_provider\_id, and created\_at. The data is as follows:

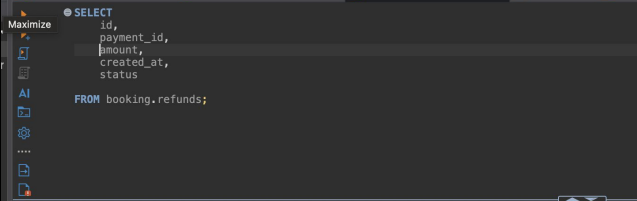
|    | id | payer_id | payee_id | booking_id | amount | status_id | payment_method_id | payment_provider_id | created_at            |
|----|----|----------|----------|------------|--------|-----------|-------------------|---------------------|-----------------------|
| 1  | 1  | 1        | 1        | 1          | 440    | 3         | 1                 | 1                   | 2026-02-22 23:15:25.8 |
| 2  | 2  | 2        | 2        | 2          | [NULL] | 1         | 2                 | 2                   | 2026-02-22 23:15:25.8 |
| 3  | 3  | 3        | 3        | 3          | 360    | 10        | 3                 | 1                   | 2026-02-22 23:15:25.8 |
| 4  | 4  | 4        | 4        | 4          | 280    | 4         | 4                 | 3                   | 2026-02-22 23:15:25.8 |
| 5  | 5  | 5        | 5        | 5          | 600    | 3         | 1                 | 6                   | 2026-02-22 23:15:25.8 |
| 6  | 6  | 6        | 6        | 6          | 75     | 2         | 5                 | 12                  | 2026-02-22 23:15:25.8 |
| 7  | 7  | 7        | 7        | 7          | 520    | 3         | 1                 | 1                   | 2026-02-22 23:15:25.8 |
| 8  | 8  | 8        | 8        | 8          | 700    | 11        | 2                 | 9                   | 2026-02-22 23:15:25.8 |
| 9  | 9  | 9        | 9        | 9          | 240    | 10        | 6                 | 18                  | 2026-02-22 23:15:25.8 |
| 10 | 10 | 10       | 10       | 10         | 880    | 3         | 4                 | 1                   | 2026-02-22 23:15:25.8 |
| 11 | 11 | 11       | 11       | 11         | 300    | 5         | 7                 | 20                  | 2026-02-22 23:15:25.8 |
| 12 | 12 | 12       | 12       | 12         | 130    | 9         | 8                 | 8                   | 2026-02-22 23:15:25.8 |
| 13 | 13 | 13       | 13       | 13         | [NULL] | 15        | 9                 | 9                   | 2026-02-22 23:15:25.8 |
| 14 | 14 | 14       | 14       | 14         | 260    | 4         | 10                | 10                  | 2026-02-22 23:15:25.8 |
| 15 | 15 | 15       | 15       | 15         | 160    | 6         | 11                | 11                  | 2026-02-22 23:15:25.8 |
| 16 | 16 | 16       | 16       | 16         | 390    | 3         | 12                | 18                  | 2026-02-22 23:15:25.8 |
| 17 | 17 | 17       | 17       | 17         | 450    | 7         | 13                | 13                  | 2026-02-22 23:15:25.8 |
| 18 | 18 | 18       | 18       | 18         | 115    | 14        | 14                | 14                  | 2026-02-22 23:15:25.8 |
| 19 | 19 | 19       | 19       | 19         | [NULL] | 16        | 15                | 16                  | 2026-02-22 23:15:25.8 |
| 20 | 20 | 20       | 20       | 20         | 500    | 3         | 16                | 17                  | 2026-02-22 23:15:25.8 |

# Entity: refunds

- **Purpose:**  
Specialized table with data on refunds.

```
• CREATE TABLE booking.refunds (  
  id INT NOT NULL,  
  payment_id INT NOT NULL,  
  amount NUMERIC NOT NULL,  
  created_at TIMESTAMP,  
  status VARCHAR,  
  PRIMARY KEY (id),  
  FOREIGN KEY (payment_id) REFERENCES booking.payments(id)  
);
```

## Test Case

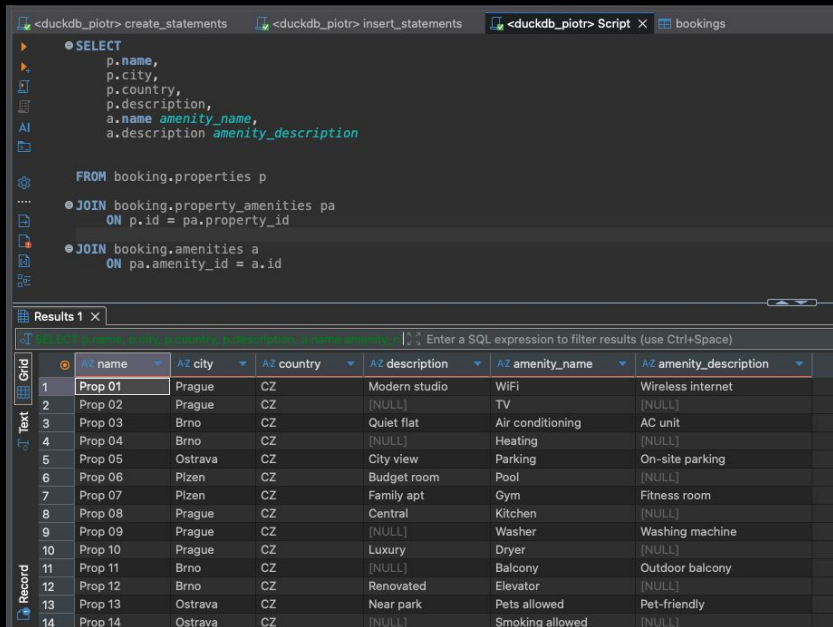


The screenshot shows a SQL IDE with a query editor at the top containing a SELECT statement. Below the editor, the results are displayed in a table grid. The table has 7 columns: an index, id, payment\_id, amount, created\_at, and status. The data consists of 20 rows, each representing a refund record. The status of the refunds varies, including 'processed', 'pending', and 'failed'.

|    | id | payment_id | amount | created_at              | status    |
|----|----|------------|--------|-------------------------|-----------|
| 1  | 1  | 1          | 50     | 2026-02-22 23:15:25.599 | processed |
| 2  | 2  | 2          | 20     | 2026-02-22 23:15:25.599 | pending   |
| 3  | 3  | 3          | 30     | 2026-02-22 23:15:25.599 | processed |
| 4  | 4  | 4          | 40     | 2026-02-22 23:15:25.599 | failed    |
| 5  | 5  | 5          | 60     | 2026-02-22 23:15:25.599 | processed |
| 6  | 6  | 6          | 10     | 2026-02-22 23:15:25.599 | pending   |
| 7  | 7  | 7          | 55     | 2026-02-22 23:15:25.599 | processed |
| 8  | 8  | 8          | 70     | 2026-02-22 23:15:25.599 | pending   |
| 9  | 9  | 9          | 25     | 2026-02-22 23:15:25.599 | processed |
| 10 | 10 | 10         | 90     | 2026-02-22 23:15:25.599 | pending   |
| 11 | 11 | 11         | 15     | 2026-02-22 23:15:25.599 | processed |
| 12 | 12 | 12         | 35     | 2026-02-22 23:15:25.599 | pending   |
| 13 | 13 | 13         | 45     | 2026-02-22 23:15:25.599 | processed |
| 14 | 14 | 14         | 65     | 2026-02-22 23:15:25.599 | pending   |
| 15 | 15 | 15         | 12     | 2026-02-22 23:15:25.599 | processed |
| 16 | 16 | 16         | 22     | 2026-02-22 23:15:25.599 | pending   |
| 17 | 17 | 17         | 32     | 2026-02-22 23:15:25.599 | processed |
| 18 | 18 | 18         | 42     | 2026-02-22 23:15:25.599 | pending   |
| 19 | 19 | 19         | 18     | 2026-02-22 23:15:25.599 | processed |
| 20 | 20 | 20         | 80     | 2026-02-22 23:15:25.599 | pending   |

# Cross-table relationships

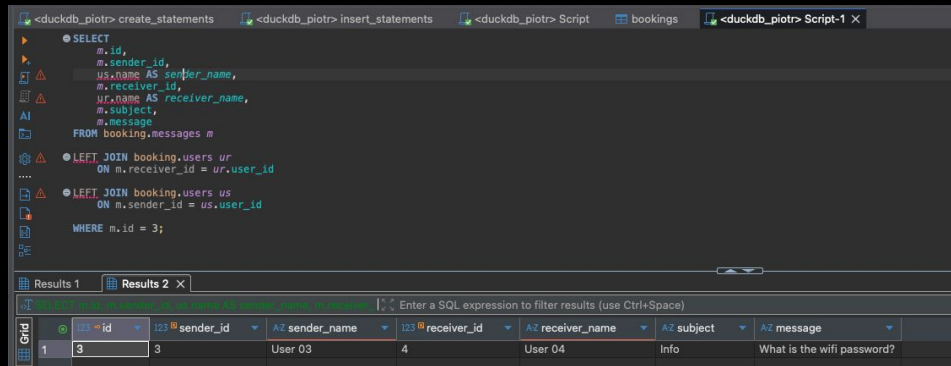
- Verification of join conditions on select tables:



The screenshot shows a DuckDB SQL editor with a query that joins properties, amenities, and booking information. The results table below shows 14 rows of data.

```
SELECT
  p.name,
  p.city,
  p.country,
  p.description,
  a.name amenity_name,
  a.description amenity_description
FROM booking.properties p
JOIN booking.property_amenities pa
  ON p.id = pa.property_id
JOIN booking.amenities a
  ON pa.amenity_id = a.id
```

|    | A? name | A? city | A? country | A? description | A? amenity_name  | A? amenity_description |
|----|---------|---------|------------|----------------|------------------|------------------------|
| 1  | Prop 01 | Prague  | CZ         | Modern studio  | WiFi             | Wireless internet      |
| 2  | Prop 02 | Prague  | CZ         | [NULL]         | TV               | [NULL]                 |
| 3  | Prop 03 | Brno    | CZ         | Quiet flat     | Air conditioning | AC unit                |
| 4  | Prop 04 | Brno    | CZ         | [NULL]         | Heating          | [NULL]                 |
| 5  | Prop 05 | Ostrava | CZ         | City view      | Parking          | On-site parking        |
| 6  | Prop 06 | Pízen   | CZ         | Budget room    | Pool             | [NULL]                 |
| 7  | Prop 07 | Pízen   | CZ         | Family apt     | Gym              | Fitness room           |
| 8  | Prop 08 | Prague  | CZ         | Central        | Kitchen          | [NULL]                 |
| 9  | Prop 09 | Prague  | CZ         | [NULL]         | Washer           | Washing machine        |
| 10 | Prop 10 | Prague  | CZ         | Luxury         | Dryer            | [NULL]                 |
| 11 | Prop 11 | Brno    | CZ         | [NULL]         | Balcony          | Outdoor balcony        |
| 12 | Prop 12 | Brno    | CZ         | Renovated      | Elevator         | [NULL]                 |
| 13 | Prop 13 | Ostrava | CZ         | Near park      | Pets allowed     | Pet-friendly           |
| 14 | Prop 14 | Ostrava | CZ         | [NULL]         | Smoking allowed  | [NULL]                 |



The screenshot shows a DuckDB SQL editor with a query that joins booking, users, and messages tables. The results table below shows 1 row of data.

```
SELECT
  m.id,
  m.sender_id,
  us.name AS sender_name,
  m.receiver_id,
  ur.name AS receiver_name,
  m.subject,
  m.message
FROM booking.messages m
LEFT JOIN booking.users ur
  ON m.receiver_id = ur.user_id
LEFT JOIN booking.users us
  ON m.sender_id = us.user_id
WHERE m.id = 3;
```

|   | m? id | m? sender_id | A? sender_name | m? receiver_id | A? receiver_name | A? subject | A? message                 |
|---|-------|--------------|----------------|----------------|------------------|------------|----------------------------|
| 1 | 3     | 3            | User 03        | 4              | User 04          | Info       | What is the wifi password? |